TERNARY SoC EMULATOR

Technical & Architectural Documentation

Implementation Language	Java 17+ / JavaFX
CPU Architecture	Balanced Ternary, Load/Store
Machine Word Width	12 trits (trit-12)
RAM Memory Size	531,441 cells (from -265720 to +265720)
Documentation Date	06/02/2026

This project implements a complete simulation of a 12-trit balanced ternary processor, along with an inline assembler and interactive graphical debugger. The system operates entirely on symmetric ternary base mathematics instead of traditional binary, supporting a comprehensive set of arithmetic, memory, and flow-control instructions.

TABLE OF CONTENTS

1. PROJECT OVERVIEW

1.1 Objective and Scope

The Ternary SoC Emulator project delivers a software simulation of a structural hardware core built entirely on balanced ternary arithmetic. Unlike standard binary devices operating on bit states (0 and 1), this processor manipulates trits (ternary digits) which support three native symmetrical values: {-1, 0, +1}.

The technical scope encompasses:

• A system core (SoC) featuring an instruction set of 27 unique operations,

• Symmetric balanced RAM supporting addressing from -265,720 to +265,720,

• An inline structural assembler compiling raw instruction mnemonics into native RAM cells,

• A JavaFX desktop interface featuring step-by-step graphical execution debugging,

• Real-time visual monitoring grids translating trit values into intuitive color block chains,

• Fault logs, operational event logging, breakpoints, and source text file mapping.

1.2 System Architecture — Layer Diagram

System Layer	Component File	Architectural Role
Presentation	GUIMain.java	JavaFX UX Layout: Workspace, monitoring grids, and execution debugger.
Logics & Translation	Interpreter.java	Source sanitation, syntax checks, and binary RAM injection.
Hardware Modeling	SoC.java	Hardware emulation: Processing core, RAM grid, and CPU instruction cycle.
System Utilities	SystemInfo.java	Environment checks: Java Virtual Machine and active OpenJFX dependencies.
App Entry	Main.java	Command-line workspace container (currently bypassed to load GUI directly).

1.3 File and Package Structure

All functional classes are packed under the javaproject workspace namespace.

Filename	Class Identifier	Primary Component Responsibility
SoC.java	SoC	Defines hardware memory cells, operational registers, and ALU processing logic.
Interpreter.java	Interpreter	Acts as the translation compiler bridge parsing raw text strings to memory words.

Filename	Class Identifier	Primary Component Responsibility
GUIMain.java	GUIMain	Initializes the application window and binds UX fields to the processing engine.
Main.java	Main	Entry stub holding CLI diagnostic loops and application launchers.
SystemInfo.java	SystemInfo	Retrieves software stack metadata values from the current JVM runtime.

2. FOUNDATIONS OF BALANCED TERNARY ARITHMETIC

2.1 What is a Trit?

A trit (ternary digit) represents the basic unit of hardware logic inside the system. Unlike conventional computers, it maps to three symmetrical physical states:

Symbol	Numeric Scalar Value	Corresponding Color Block Rendering in UX
+	+1 (Positive State)	Green Fill Block (#238636)
0	0 (Neutral State)	Gray Fill Block (#30363d)
-	-1 (Negative State)	Red Fill Block (#da3633)

2.2 Number Representation — Range and Encoding

A standard hardware register spans a functional width of 12 trits. This maps to signed integer values in the symmetric bounded range of [-265,720, +265,720]. The maximum boundaries are derived directly via base-3 exponential constraints:

```
max_value = (3^12 - 1) / 2 = (531,441 - 1) / 2 = 265,720
```

The internal method truncate12(value) processes any arithmetic result into this bounded field, realizing a stable modular balanced wrapping wrap:

```
mod_result = (value + 265720) % 531441
final_bounded_value = mod_result - 265720 (symmetric range shift correction)
```

2.3 Mathematical Properties

• Perfect symmetry around zero — completely avoids the double-zero artifacts and sign-extension complexities found in standard binary Two's Complement systems.

• The neutral state (0) allows native data representation of unassigned values or idle states.

• Trit shifting (SHF) scales values cleanly by scaling factors matching base-3 powers.

• Overflow tracking is symmetrical, isolating positive overflow bounds (+1) from negative ones (-1).

3. THE SoC.java MODULE — PROCESSOR CORE

The SoC file contains the primary virtual hardware model. It allocates memory arrays acting as the RAM grid, manages data tracks, defines structural operational registers, and drives the fetch-decode-execute processing loops.

3.1 Hardware Registers

Mnemonic	Bit/Trit Width	Primary Architectural Responsibility	Boot/Reset Default State
PC	12 trits	Program Counter — Tracks memory addresses of current steps.	START_ADDRESS = -265,720
SP	12 trits	Stack Pointer — Allocates memory ranges for hardware call paths.	MAX_VALUE = +265,720
R-	12 trits	General purpose data register (Symmetric index match: -1).	0
R0	12 trits	Main system Accumulator workspace (Symmetric index match: 0).	0
R+	12 trits	General purpose data register (Symmetric index match: +1).	0
STS	1 trit	Status Flag — Logs output characteristics (-1=Neg, 0=Zero, +1=Pos).	0
OVR	1 trit	Overflow Flag — Identifies math boundary crossing (-1 / 0 / +1).	0
HLT	boolean	System State Flag — Pauses operations when flag hits true.	false

3.2 RAM Memory Organization

The physical RAM structure consists of an integer array totaling 531,441 cells, indexed logical-to-physical across boundaries spanning [-265,720 to +265,720]. Critically, each individual cell holds 6 trits (exactly one half-word slice of a standard 12-trit machine parameter).

Mathematical mapping of logic addresses to array indexes:

```
array_index = (logical_address + MAX_VALUE) % ramSize
```

12-trit operations (LDR, STR, PSH, POP, CALL, RET) automatically span across two adjacent 6-trit RAM rows to process values cleanly:

```
writeRam12(address) → RAM[address] = upper_half, RAM[address+1] = lower_half
```

3.3 Instruction Word Format

Every assembly operation populates exactly two continuous cells inside the RAM grid (2 × 6 trits = 12 trits):

Memory Slot	Assigned Word Name	Internal Bitwise/Tritwise Layout Breakdown
RAM[PC]	Command Word	[Unused / 0][Opcode (3 Trits)][Target Rd (1 Trit)][Source Rs (1 Trit)]
RAM[PC+1]	IMM Data Word	Holds a 6-trit Immediate constant parameter (Integer field matching [-364, +364])

The packed structural Command Word is calculated via: cmdWord = (opcodeId * 9) + (rd * 3) + rs, where opcodeId maps to [-13, +13], rd tracks destination codes □ {-1, 0, +1}, and rs tracks source codes □ {-1, 0, +1}.

3.4 Instruction Cycle (fetch-decode-execute)

Step ID	Target Operational Task	Simulated Underlying Logic Command (Java-based)
1. Fetch Command	Extract operational instruction code from current PC index row.	cmdWord = readRam(this._programcounter);
2. Fetch Data	Extract inline immediate data from subsequent layout row.	imm = readRam(this._programcounter + 1);
3. Advance PC	Shift Program Counter forward to align with the next block.	_programcounter += 2;
4. Shift Base	Normalize raw packed command numbers out of negative loops.	shifted = cmdWord + 364;
4a. Extract Rs	Isolate source register indices via base-3 modulo math.	rs = (shifted % 3) - 1;
4b. Extract Rd	Isolate destination register parameters via base-3 scaling.	rd = ((shifted / 3) % 3) - 1;
4c. Extract Op	Determine the target instruction identifier index code.	opcodeId = ((shifted / 9) % 27) - 13;
5. Execute Steps	Pass extracted pointers down to execution routing blocks.	executeDecoded(opcode, rd, rs, imm);

3.5 CPU Status and Control Flags

Flag Name	Trit Width	Native Values & Conditional Logic State	Triggering Operations Pool
STS (Status)	1 trit	+1 = Output > 0 | 0 = Output == 0 | -1 = Output < 0	ADD, SUB, ADC, SBC, CMP, MIN, MAX, INV, SHF
OVR (Overflow)	1 trit	+1 = Positive limit crossed | 0 = Normal math | -1 = Negative limit crossed	Evaluated by all ALU components via updateFlags()
HLT (Halt)	boolean	true = Operations loop suspended; processing halts safely	Triggered directly via HLT command

3.6 Complete Instruction Opcode Set

Internal decoder array mapping for the hardware execution grid:

Instruction	ID Code	Instruction	ID Code
SUB	-13	SHF	-12
ADD	-11	SBC	-10
CMP	-9	ADC	-8
MIN	-7	INV	-6
MAX	-5	LDR	-4
LLI	-3	LHI	-2
STR	-1	MOV	0
PSH	1	POP	2
PSF	3	POF	4
JMN	5	OVN	6
JMP	7	WSP	8
JMZ	9	HLT	10
CALL	11	OVP	12
RET	13	—	—

4. THE INTERPRETER.JAVA MODULE — ASSEMBLER

The Interpreter class performs the structural compilation phase. It takes raw text strings from the source window, purifies formatting, evaluates syntax boundaries, and builds binary machine words directly inside the RAM target cell blocks.

4.1 Parsing Phase and Code Clean-up

The static optimization method optimize(String line) filters inputs as follows:

- Strips out inline user notation tags (anything tracking trailing behind double slashes //),
- Trims edge whitespaces and trailing structural returns from terminal buffers,
- Condenses multiple spacing gaps inside parameters into singular space markers via regex matching.

4.2 Syntax Validation Rules

The compiler tracking loop validateSyntax(String[] parts) evaluates parameter sizes against token validation collections:

Collection Group	Assigned Instructions Pool	Required Operational Parameter Layout
NO_ARG	HLT, PSF, POF, RET	Strictly 0 extra arguments (opcode identifier string only)
RD_ONLY	INV, PSH, POP, JMN, OVN, JMP, WSP, CALL, OVP, JMZ	Exactly 1 target parameter (Target destination register Rd)
RD_RS	SUB, SHF, ADD, SBC, CMP, ADC, MIN, MAX, LDR, STR, MOV	Exactly 2 parameters (Destination Rd, followed by Source Rs)
RD_IMM	LLI, LHI	Exactly 2 parameters (Destination Rd, followed by Immediate scalar data integer)

4.3 Instruction Encoding to RAM

The execution logic method assembleAndLoad(List programLines) walks code inputs and commits binary data down to memory blocks:

1. Evaluates optimize() loops → returns a sanitized command string.
2. Intercepts memory mapping indicators [n] → overrides the active currentRamAddress pointer.
3. Extracts functional fields from token parameters: opcodeId, rd, rs, imm.
4. Calculates command packaging structures: commandWord = (opcodeId * 9) + (rd * 3) + rs.
5. Injects components down to RAM layers: cpu.injectRAM(addr, cmd) + cpu.injectRAM(addr+1, imm).
6. Increments the allocation counter: currentRamAddress += 2 (allocates 2 cells per operation block).

4.4 Assembler Directives

Directive Mnemonic	Syntax Pattern Example	Functional Compilation Effect
Inline Comment	// annotation text	Instructs the line scanner to ignore trailing details following double slashes.
Block Comment	# annotation text	Instructs the compiler loop to completely ignore the current row line.
RAM Origin Shift	[target_address]	Forces the assembler memory tracking counter to point directly to a new target RAM cell.
Debugger Breakpoint	* instruction_mnemonic	Flags the specified operation block, forcing runtime execution to pause before it.

5. THE GUIMain.java MODULE — GRAPHICAL INTERFACE

The GUIMain component manages the user application interface layout. It establishes an interactive, responsive configuration window built atop a clean light layout that exposes internal processor mechanics transparently.

5.1 Screen Layout and Components

Panel Block	BorderPane Slot Position	Interface Component Allocation
Action Toolbar	Top Alignment Area	Houses global macro triggers, file controls, and compilation trackers.
Left Interface Column	Left Bounded Side	Displays the live register metrics stack alongside the Stack Explorer view.
Center Workspace Window	Center Workspace Slot	Displays a real-time table tracking active memory allocations across RAM cells.
Right Configuration Column	Right Bounded Side	Houses the code terminal editor input window and the system operational history log.

5.2 Action Toolbar Layout

Toolbar Macro Element	Keyboard Short-key	Underlying Action Response Trigger
□ RESTART	—	Flushes current configurations, mounts fresh system engines, and purges logs.
? MANUAL	—	Launches an informative reference modal displaying internal hardware guidelines.
□ OPEN	Ctrl + O	Triggers a native FileChooser dialog to load external code documents into the editor workspace.
□ SAVE	Ctrl + S	Commits edited scripts directly back to storage files or routes to Save As loops.
□ Dropdown Menu Arrow	—	Exposes context options to invoke secondary tracking loops like Save As (Ctrl+Shift+S).
□ SYNTAX CHECK	—	Runs text verification processes to evaluate code lines without executing them.
□| STEP	—	Ticks processor execution loops forward to handle exactly one single operation line.
□ RUN FULL	—	Drives processing loops forward continuously until reaching halt calls or breakpoint flags.

5.3 Source Code Editor Panels

The source code entry field is an interactive TextArea component. The workspace pod features an optimized background hint label that displays a minimal code

template when empty:

```
LLI R0 10 LLI R+ 5 ADD R0 R+ *CMP R0 R+ HLT
```

5.4 Processor State Visualization

The monitoring console charts hardware status through an intuitive hybrid readout. Each internal register displays its current value as both a standard decimal string and an explicit line of 12 rendered color boxes. System pointers (PC, SP) are labeled with clear blue labels (#58a6ff), while general-purpose data blocks (R-, R0, R+) use neutral gray markings (#c9d1d9). Symmetrical flags (STS, OVR, HLT) track metrics inline with unique purple text decorations (#d2a8ff).

5.5 Stack Explorer Functionality

The Stack Explorer tracks stack allocations by reading memory cells starting at the active SP pointer index and scanning upwards across 8 rows. Because balanced ternary stacks scale downward (SP decrements during a PSH call and increments during POP execution), this monitor provides a continuous view of historical stack frames.

5.6 Live RAM Memory Mapping Window

The primary center terminal exposes memory contents across a real-time 15-cell window centered on the current PC row value. The display maps parameters in a clean grid format:

```
[RAM ADDRESS LOCATION] | DECIMAL SCALAR | [6-TRIT TERNARY GRAPHIC CHAINS] << PC | [B]
```

The prominent marker << PC highlights the active instruction row, while the indicator tag [B] identifies committed breakpoint lines.

5.7 System Event Logging

The lower log window tracks all internal operations and errors. It renders detailed statuses, including compilation successes, inline typos `[ERROR Ln]`, critical out-of-bounds indicators `[KERNEL PANIC]`, or instruction pauses `[BREAKPOINT]`.

5.8 Debugger — Breakpoints and Stepping Modes

The compiler intercepts the leading asterisk character *, mapping its instruction index into a lookup set (`breakpointsAddrs`). The execution router evaluates operations using two separate runtime branches:

Debugging Mode	Internal Method Flags	Runtime Execution Characteristics
STEPPING (Single Ticks)	stepMode = true	Executes precisely one instruction frame before halting safely (sets maxExecutionLimit to 1).

Debugging Mode	Internal Method Flags	Runtime Execution Characteristics
RUN FULL (Continuous)	stepMode = false	Drives processing loops forward continuously until hitting an internal HLT call, intercepting an active breakpoint address, or exceeding a 1,500-cycle guard limit.

The internal state tracking latch runPaused lets users resume execution after hitting a breakpoint without having to clear memory or reload the script.

5.11 Embedded System Manual Modals

The integrated help dialog launches a TabPane interface containing four structured subsections designed to guide programming workflows:

• Quick Start — High-level tutorial introduction presenting color profiles and standard execution workflows.

• Architecture — Technical overview outlining machine word structures, split-cell memory layouts, and register tracks.

• Instructions — Reference catalog summarizing operations, tracking flag adjustments, and detailed command profiles.

• Examples — Pre-configured functional routines including calculations, memory testing pipelines, and standard Fibonacci sequence loops.

6. THE MAIN.java MODULE — APPLICATION ENTRY

The Main class serves as the initial application entry point. It retains a historical console-based CLI loop inside block comments, which originally allowed manual instruction stepping and terminal reporting. In production configurations, this CLI wrapper is bypassed to route application execution directly into the desktop setup via GUIMain.main().

7. THE SystemInfo.java MODULE — ENVIRONMENT UTILITY

The SystemInfo utility class provides diagnostic functions to query current workspace dependencies at runtime. It exposes two main environment query methods:

Static Method Identifier	Returned Diagnostic Environment String Token
javaVersion()	Fetches the current runtime environment string value from System.getProperty("java.version").
javafxVersion()	Fetches the active OpenJFX dependencies layout string token from System.getProperty("javafx.version").

8. ASSEMBLY INSTRUCTIONS — FULL SPECIFICATION

8.1 Arithmetic and Logic Unit (ALU) Operations

Mnemonic	Syntax Layout	Core Hardware Mathematical Action	Updated Flags
ADD	ADD Rd Rs	Rd = Rd + Rs (Performs signed addition)	STS, OVR
SUB	SUB Rd Rs	Rd = Rd - Rs (Performs signed subtraction)	STS, OVR
ADC	ADC Rd Rs	Rd = Rd + Rs + OVR (Includes incoming overflow)	STS, OVR
SBC	SBC Rd Rs	Rd = Rd - Rs + OVR (Subtracts value and injects overflow)	STS, OVR
CMP	CMP Rd Rs	Evaluates STS = sign(Rd - Rs). Target register states remain completely untouched	STS, OVR
MIN	MIN Rd Rs	Rd = min(Rd, Rs) (Loads the minimum scalar match value)	STS, OVR
MAX	MAX Rd Rs	Rd = max(Rd, Rs) (Loads the maximum scalar match value)	STS, OVR
INV	INV Rd	Rd = -Rd (Inverts all trit components symmetrically)	STS, OVR
SHF	SHF Rd Rs	Scales trit structures by base-3 values: Multiplies when Rs > 0, divides when Rs < 0	STS, OVR

8.2 Memory and Register Manipulation Instructions

Mnemonic	Syntax Layout	Functional Hardware Processing Action
LLI	LLI Rd IMM	Loads Immediate constant values into the lower 6 trits of register Rd; upper row states are preserved.
LHI	LHI Rd IMM	Loads Immediate constant values into the higher 6 trits of register Rd; lower row states are preserved.
LDR	LDR Rd Rs	Loads a full 12-trit machine parameter from the combined RAM address cell indexed inside Rs.
STR	STR Rd Rs	Stores the full 12-trit value from register Rd down into the double-cell memory grid indexed by Rs.
MOV	MOV Rd Rs	Copies data directly from register Rs into destination register Rd.

Architectural Note: Loading a full 12-trit address value requires a paired configuration using both LHI and LLI operations. For instance, setting register R0 to hold the value 1000 is written as:

```
LHI R0 1 // Configures higher block metrics: 1 * 729 = 729
LLI R0 271 // Configures lower block parameters: + 271 = 1,000
```

8.3 Stack Push and Pop Operations

Mnemonic	Syntax Layout	Functional Hardware Stack Interaction Step
PSH	PSH Rd	Decrements the Stack Pointer row and pushes a 12-trit data frame onto the stack: SP -= 2; RAM[SP] = Rd.
POP	POP Rd	Pops a 12-trit frame off the stack and copies it into register Rd, then advances the pointer: Rd = RAM[SP]; SP += 2.
PSF	PSF	Pushes the current flag register values onto the stack using a packed format: packed = (STS * 3) + OVR.
POF	POF	Pops a packed flag frame off the stack, restores status bits, and handles sign corrections cleanly.
WSP	WSP Rd	Forces the Stack Pointer to target the address currently held inside register Rd: SP = Rd.

8.4 Flow Control and Jumps

Mnemonic	Syntax Layout	Required Hardware Latch Flag Bounds	Underlying Branch Routing Action
JMP	JMP Rd	Status Latch sets positive state (STS == 1)	Routes Program Counter to match address target: PC = Rd
JMN	JMN Rd	Status Latch sets negative state (STS == -1)	Routes Program Counter to match address target: PC = Rd
JMZ	JMZ Rd	Status Latch sets neutral state (STS == 0)	Routes Program Counter to address target: PC = Rd
OVP	OVP Rd	Overflow Latch sets positive state (OVR == 1)	Routes Program Counter to match address target: PC = Rd
OVN	OVN Rd	Overflow Latch sets negative state (OVR == -1)	Routes Program Counter to match address target: PC = Rd
CALL	CALL Rd	Unconditional execution routing	Pushes the current return step onto stack layers, then routes PC to Rd
RET	RET	Unconditional execution routing	Pops the return step address off stack layers back into PC

8.5 System Halt Directives

Mnemonic	Syntax Layout	Functional System Architecture Execution Effect
HLT	HLT	Sets the hardware termination flag HLT to true. This suspends all processing loops and safe-locks the engine.

9. CODE EXAMPLES AND USE CASES

9.1 Demonstrating Conditional Jumps

This sample program subtracts two equal values and uses the JMZ instruction to jump over an endless operational path, terminating execution safely.

```
LLI R0 5 // Main Accumulator Register R0 = 5
LLI R+ 5 // Auxiliary Register R+ = 5
SUB R0 R+ // Math task: R0 = R0 - R+ = 0 (Triggers Status Flag to hit 0)
LLI R- -265710 // Load safe exit target address into register R-
JMZ R- // Evaluates condition: Jump to R- if status is zero (Routes loop to exit address)
HLT // Bypassed processing row (will not be read)
[-265710] HLT // Directives row: Secure hardware termination endpoint
```

9.2 Absolute Addressing and Directives

This application shows how to assign raw mapping zones inside RAM layers using the brackets indicator option:

```
[-250000] // Sets current compilation allocation row to index -250000
LLI R0 42 // Load numeric constant value 42 into register R0
LLI R- 250 // Set target address memory marker pointer R- = 250
STR R0 R- // Memory Action: Commit content of R0 down to cell RAM[250]
LDR R+ R- // Memory Action: Read cell content from RAM[250] back into register R+
HLT // Terminate system execution loop cleanly
```

9.3 Fibonacci Sequence Loop Generation

This program computes consecutive Fibonacci sequence values, utilizing the R+ register as an iteration counter. Upon execution, R- and R0 will contain two adjacent Fibonacci numbers.

```
LLI R- 0 // F(0) = 0
LLI R0 1 // F(1) = 1
LLI R+ 8 // Iteration counter
PSH R+ // Push counter to the stack
[-265712] PSH R0 // [loop target] Push current R0 to stack
ADD R0 R- // R0 = R0 + R- (compute next Fibonacci number)
POP R- // Old R0 -> R-
POP R+ // Retrieve counter
PSH R0 // Push new R0 safely
LLI R0 1 // Temporarily load 1 to R0
SUB R+ R0 // Decrement iteration counter
POP R0 // Restore R0
PSH R+ // Push updated counter to stack
LLI R+ -265712 // Load loop target address into R+
JMP R+ // Jump to loop target if counter > 0
POP R+ // Clean up stack when loop finishes
HLT
```

10. SAFETY LAYERS AND RUNTIME FAULT INTERCEPTION

Hardware Boundary Monitor	Triggering Condition Criteria	System Error Resolution Response
Watchdog Step Limit	stepsExecuted >= 1500 during a single RUN execution loop	Halts execution loops safely and updates log: [TIMEOUT]
Stack Guard Check	SP <= START_ADDRESS (-265,720) bound limits crossed	Halts execution and updates terminal log: [KERNEL PANIC]
Program Counter Guard	PC location ventures beyond limits [-265,720 to +265,720]	Blocks operations immediately and logs: [KERNEL PANIC]
Decoder Exception Catch	opcodeId can not be resolved via active OPCODE_MAP keys	Halts the engine loop and throws a RuntimeException
Syntax Typo Catch	validateSyntax() execution detects a format mismatch	Logs explicit line error `[ERROR Ln]` and skips row compilation

11. TECHNICAL REQUIREMENTS AND VERIFICATION CONSTRAINTS

Architectural Component	Minimum Required Specification Version	Target Operational Verification Notes
Java Development Kit	Java SE SDK 17 or higher release layers	Provides base runtime support for module configurations.
JavaFX Dependency SDK	OpenJFX Version 21.0.2 core libraries	Requires controls and graphics dependencies to run properly.
Operating System Environment	Windows Desktop / Linux Systems / Apple macOS	Provides seamless platform portability via optimized JVM threads.

Documentation generated automatically for the Ternary SoC Emulator. Build Date: 06/02/2026 | Project Target: Ternary SoC Emulator | Namespace: javaproject